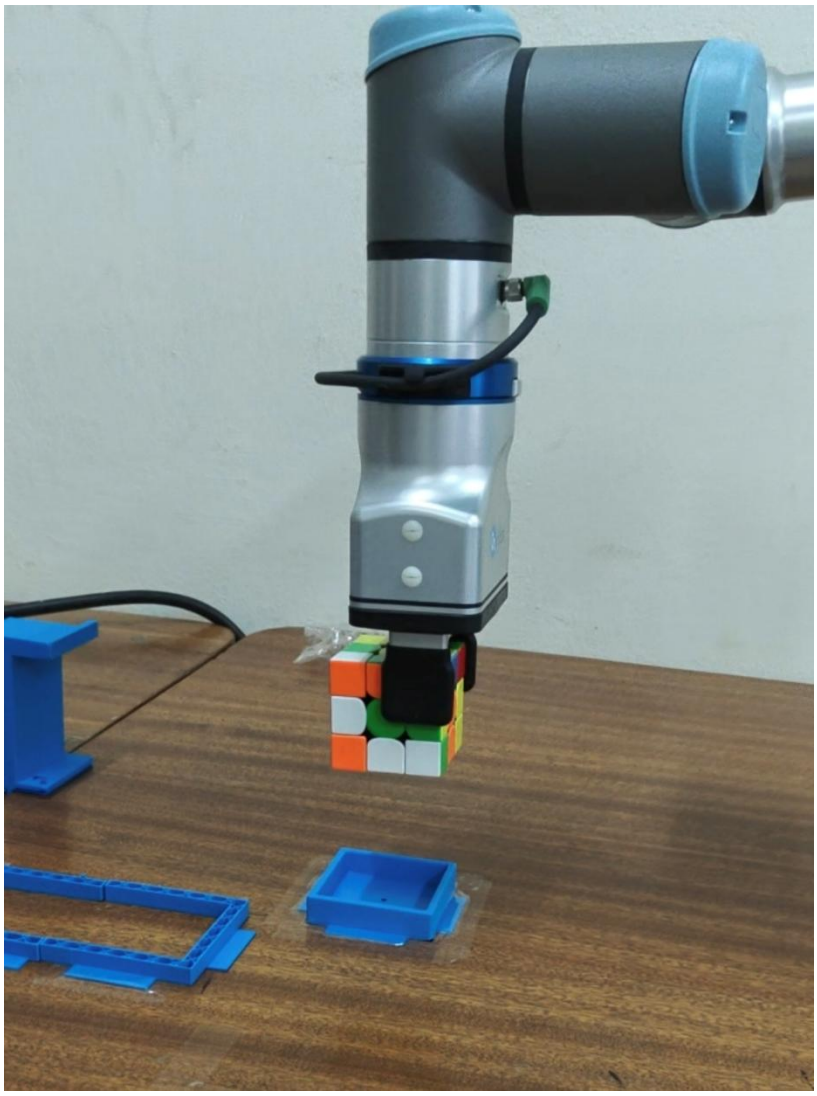
School Lab Work :

Rubik’s Cube resolution using Python



Summary

SUMMARY 2

INTRODUCTION AND CONTEXT **3**

1. Mini-Project Objectives and Organization 3
2. System Overview and Components 3
3. Introduction to the UR-Script Command Language 4
4. Code Architecture 4
5. Introduction to Cube Movement by the Robot 5

PROGRAMMING SESSION **6**

1. Step 1 : Memory 6
2. Step 2 : Robot movement 8
3. Step 3 : Cube resolution 13
4. Step 4 : Script Generation and File Export 15

TESTING SESSION **16**

1. Code 16
2. Robot 17

TO GO FURTHER **20**

Introduction and Context

1. Mini-Project Objectives and Organization

This cycle of practical lab work is designed as an open technological integration workshop. The objective is strictly formative and oriented towards self-directed learning: the goal is to design, implement, and test an industrial robotics software application capable of solving a physical Rubik's Cube. To encourage experimentation, algorithmic risk-taking, and technical discovery, this workshop requires no report submissions, has no mandatory deliverables, and will not be subject to any grading.

In addition to this written guide, a series of video demonstrations will be provided for you. These videos will illustrate exactly how the robotic arm physically executes each specific movement (such as standard face rotations, double rotations, and whole-cube reorientations). This visual support is designed to help you better grasp the kinematic constraints of the robot and easily bridge the gap between abstract algorithms and physical reality.

You are invited to progress at your own pace through the following topics:

- Geometric and spatial analysis of a moving solid manipulated by a robotic gripper.
- Functional specification of tool space coordinate transformations.
- Python programming of a generator capable of translating abstract solutions into physical trajectories.
- Integration of safety and diagnostic modules to operate the physical robotic arm with full confidence.

2. System Overview and Components

The system to be controlled is a precision robotic manipulation cell integrating several key industrial components.

At the core of the setup is a collaborative robotic arm (Universal Robots). This 6-degree-of-freedom articulated robot offers high repeatability and can be programmed using both joint and linear trajectories. Mounted on its wrist is an adaptive two-finger electric gripper, which allows the robot to firmly grasp the faces of the standard Rubik's Cube (the scrambled object to be solved). The gripper's opening width is dynamically adjustable through the code to accommodate different manipulation steps.

Within the robot's workspace sits a fixed centering stand. This passive mechanical device is crucial to the process, serving as an absolute spatial reference. It allows the

robot to temporarily place the cube down, release the gripper, change its approach axis, and pick the cube back up. This sequence is essential for rotating faces that are initially inaccessible to the arm.

Finally, a PC development station (running Python 3 via the Spyder IDE) acts as the system's logic hub. This workstation is used to program the application's intelligence and generate the command files, which are then transferred to the robot controller via a USB drive.

3. Introduction to the UR-Script Command Language

The Universal Robots robotic arm controller uses a proprietary, low-level, trajectory-oriented language called UR-Script. Your Python application's role will be to generate a .script file containing a sequential series of UR-Script geometric primitives.

The movement and actuator functions we will use from the provided macro dictionary are:

- $movel(p[x, y, z, rx, ry, rz], a, v)$: Linear movement of the tool to the specified Cartesian point (in meters and radians), with an acceleration a (m/s^2) and a velocity v (m/s).
- $twofg_grip(width, force, speed)$: Controls the opening of the gripper (width in mm; for example, 40 to grip the central slice of the cube, 80 to completely release the object).

4. Code Architecture

The structural backbone of the Python application is divided into four main functional components, designed to decouple the abstract algorithmic logic from the physical robotic execution:

- **1. Spatial Memory:** This module continuously tracks the orientation of the cube within the workstation's reference frame. It dynamically updates the face mappings whenever the robotic arm reorients or flips the entire cube, ensuring the software always knows exactly which physical side corresponds to each logical face.
- **2. Robot Kinematics and Manipulation:** This section defines physical trajectories and gripper behaviors. It handles safety clearances, controls the opening and closing width of the gripper fingers, and manages the coordination between linear (*movel*) movements for smooth physical execution.

- **3. Cube Resolution using Python:** This part handles the algorithmic intelligence of the project. It captures the initial state of the cube, translates it into the standardized 54-character string required by the kociemba library, executes the solver, and parses the raw text solution into a clean, iterable list of individual moves.
- **4. Script Generation and File Export:** The final component acts as a compiler. It loops through the parsed list of moves, pairs each action with its corresponding UR-Script commands from the macro database, and writes a fully functional, syntactically correct .script file directly onto the USB drive, ready to be executed by the robot controller.

5. Introduction to Cube Movement by the Robot

To support your technical understanding, a dedicated video demonstration is provided for each movement, allowing you to visualize the exact physical kinematics in real time of the robot arm. To be synthetic, an **R movement** is a clockwise movement of the Right face, an **L movement** is a clockwise movement from the Left face, and so on with Back, Front, Up and Down. When the letter is followed by an apostrophe, it means that the movement will be counterclockwise, ex : R' is a counterclockwise movement of the right face. Lastly when the letter is followed by 2, it means we want to do a full 180° rotation, ex : R2 is a 180° rotation of the right face.

Additionally, we will explore the specific positioning challenges encountered when attempting to grasp the cube under various configurations, as well as the critical importance of using joint movements (*movej*) in certain scenarios to prevent singularities or mechanical interference.

Finally, this section explains the vital role of the robot's spatial memory. Maintaining an accurate digital twin of the cube's orientation prevents redundant, time-consuming back-and-forth movements around a single reference position, enabling the robot to always know exactly how the cube is oriented in space.

Programming session

1. Step 1 : Memory

The first step in the process of resolving a Rubik's cube with the robot is making sure that the robot understands and knows how the cube rotates in space.

The goal here is to create one function for each different movement possible on the cube (L, R, F, B, ...). Each function will rewrite the dictionary `cube_memory` by changing the values, which are the symbolical faces, of each key, which are the physical faces in space currently.

To help see how it's built, the dictionary `cube_memory` and the first function are shown here, you can rewrite them in your script.

```
cube_memory = {
    "U": "UP",     # Face Up is on the top initially
    "D": "DOWN",   # Face Down is on the bottom initially
    "F": "FRONT", # Face Front is forward initially
    "B": "BACK",   # Face Back is backward initially
    "L": "LEFT",   # Face Left is facing the left initially
    "R": "RIGHT"   # Face Right is facing the right initially
}

def rotation_L():
    """
    All the rotation functions are parts of the memory for the robot.
    rotation_X modifies the dictionary 'cube_memory' by applying an X rotation.

    """
    global cube_memory

    # We save how the cube is currently oriented before modifying it
    previous = cube_memory.copy()

    cube_memory["L"] = previous["D"] # The previous down is now at the left
    cube_memory["R"] = previous["U"] # The previous up is now at the right
    cube_memory["B"] = previous["F"] # The previous front is now at the back
    cube_memory["F"] = previous["B"] # The previous back is now at the front
    cube_memory["U"] = previous["R"] # The previous right is now up
    cube_memory["D"] = previous["L"] # The previous left is now down
    return
```

You can now use the videos and the cube to create all the 17 remaining functions you will need to have all the possible movements registered in your code.

After doing that, you must create a new dictionary with all the different functions to be able to call them when needed. Using what is already in your code as inspiration, you can create the dictionary called `rotations`.

```
rotations = {
    "L": rotation_L,

}
```

Finally, the last step for the memory is to create a function that can automatically modify the proposed solution to include the fact that the cube rotates in space according to the base of the robot.

The function will follow the structure shown below :

```
def memorisation(initial_solution = list):
    """
    This function's purpose is to change the solution to implement a memory for the robot
    because of the fact that the robot will change the orientation of it, so the faces will
    change during the resolution of the cube.
    """
    sol = []


    return sol
```

This function will look at all the movements in the proposed solution and create a new one in 3 steps :

- Analyze the requested movement (which face ? simple/double ? clockwise ?)
- Determine the physical position to move by looking in the dictionary `cube_memory`
- Write the real movement in the new solution and apply this real movement on the dictionary `cube_memory`

Try doing this function with those steps.

2. Step 2 : Robot movement

The trickier part is to make the robot move how we want it to move.

The part of this code that you already have is to make the robot do an **L turn** clockwise. We are going to go through together :

1. Take and lift the cube :

```
"#take from above",
"movel(p[-0.335, -0.1, 0.15, 2.221, -2.221, 0], a=2, v=1)",
"movel(p[-0.335, -0.1, 0.065, 2.221, -2.221, 0], a=1.2, v=0.75)",
"twofg_grip(40, 30, 2)",
"movel(p[-0.335, -0.1, 0.15, 2.221, -2.221, 0], a=1.2, v=0.75)",
```

As it is said in the comment in the code, those lines are to take the cube from the top, so in the lines order :

- It goes on the top of the cube, but a little to the right to not clamp on the left sides
- It goes down around the cube
- It grips the cube
- It lifts the cube from the table in the same position as before

The picture shows the position of the clamps at the second step.



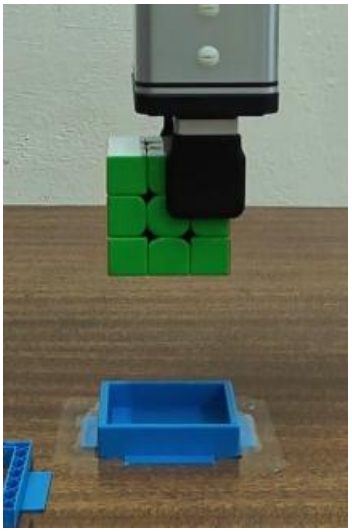
2. Rotate the left side down

```
"#place the left side down",
"movel(p[-0.351, -0.066, 0.15, 0, 2.221, -2.221], a=2, v=1)",
"movel(p[-0.351, -0.066, 0.052, 0, 2.221, -2.221], a=1.2, v=0.75)",
"twofg_grip(80, 30, 2)",
"movel(p[-0.351, -0.066, 0.15, 0, 2.221, -2.221], a=1.2, v=0.75)",
```

As it is said in the comment in the code, those lines are to move the left side down :

- It rotates the left side down
- It puts the cube down
- It releases the cube
- It lifts itself from the cube to allow further movements

The first picture shows the position before arriving to this section of the code and the second one, how the robot is oriented when putting the cube down.





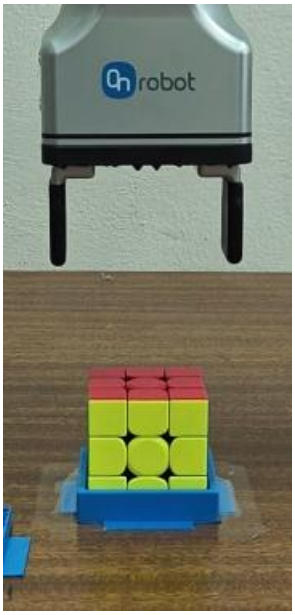
3. Moving the clamp up and rotating the face

```
"#move the clamp up",
"movel(p[-0.351, -0.1, 0.15,  0, 3.142, 0], a=2, v=1)",
"movel(p[-0.351, -0.1, 0.065, 0, 3.142, 0], a=1.2, v=0.75)",
"twofg_grip(40, 30, 2)",
"#turn the left side clockwise",
"movel(p[-0.351, -0.1, 0.067, 2.221, 2.221, 0], a=1.2, v=0.5)",
"twofg_grip(80, 30, 2)",
"movel(p[-0.351, -0.1, 0.15, 2.221, 2.221, 0], a=1.2, v=0.75)"
```

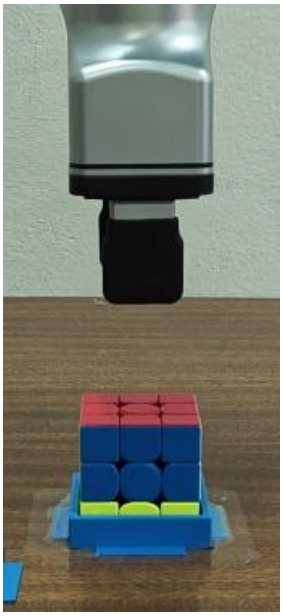
As it is said in the comment in the code, those lines are to move the clamp up and turn the left side clockwise ：

- Replace the clamp up exactly in the middle of the cube
- Moving down on the cube
- Grip the cube
- Turning the face clockwise while gripping the cube
- Releasing the cube
- Lifting the clamps from the cube

The three pictures below are respectively at the end of the first step, during the fourth step and at the end of this section.

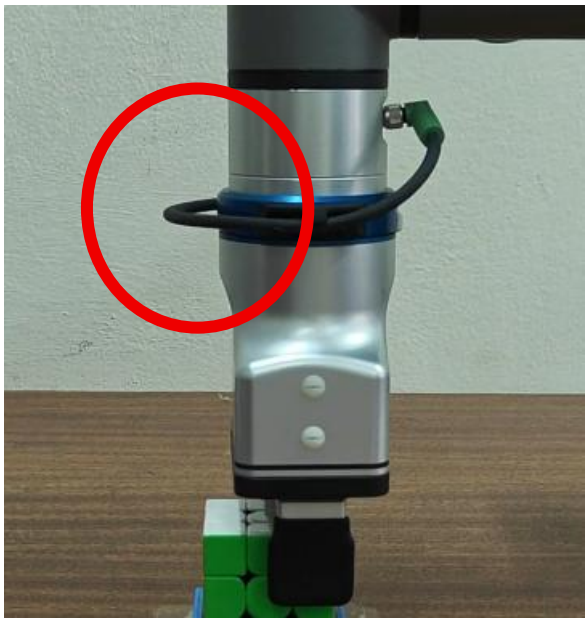




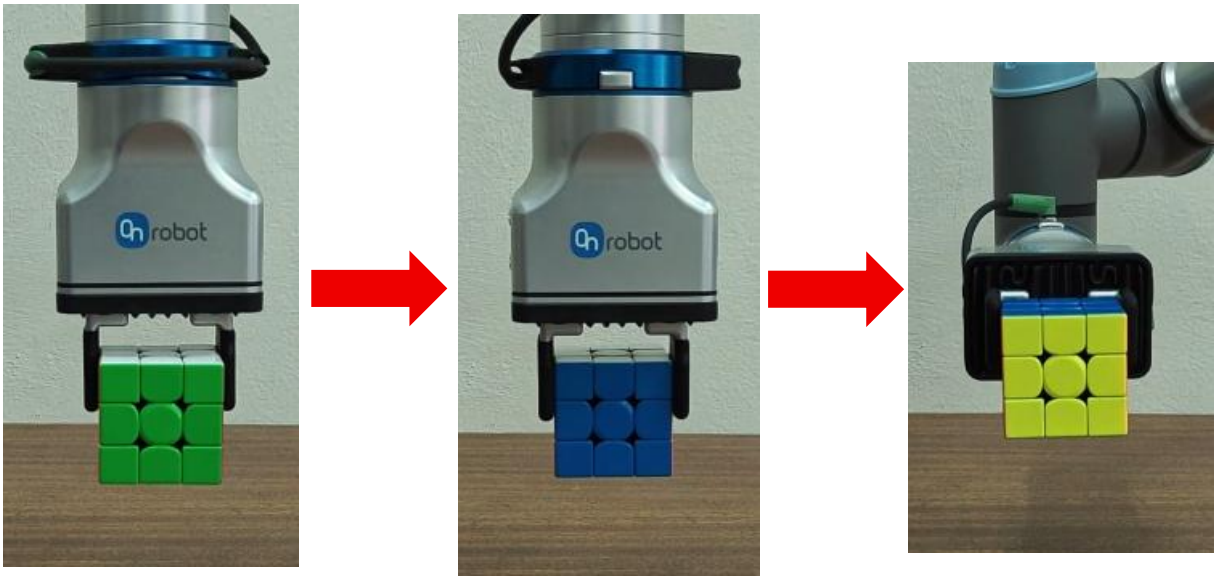


Warning : To move the cube in space without harming the cube, the robot or something else, there are some rules :

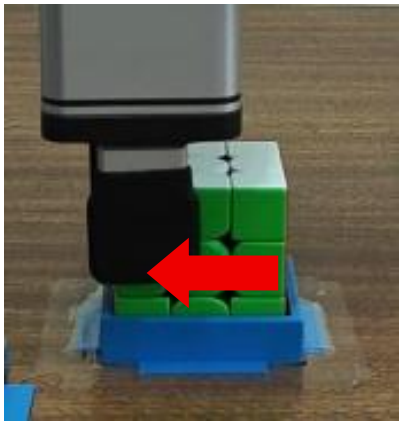
1. The loose part of the cable going from the robot arm to the tool must always face the face which will be moved. For example, in the following picture the left side will be moved so the loose part is facing the left face :

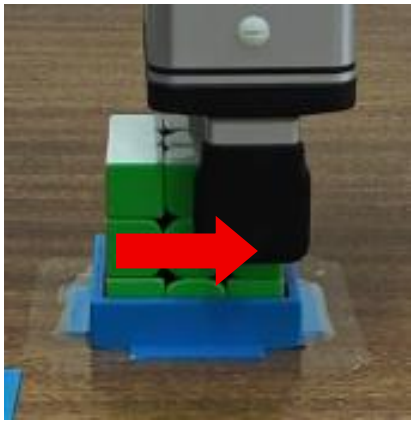


2. When you want to place the front face down, you must do it in two steps, first rotate the front face at the back then rotating it down, like shown below :



3. Like the left face, the robot must not clamp the side which is supposed to be moved below during the first step, otherwise the robot won't be able to place it in the slot on the table. Below is represented the shift regarding the middle of the cube for an **R movement** and an **L movement** :





4. Except when you are rotating a face down, you should never move the clamp below 65mm from the origin, and when you are rotating a face down you should never go below 52mm.

5. Always include the offset according to the TCP when putting the cube down, the middle of the cube is not always aligned with the TCP.

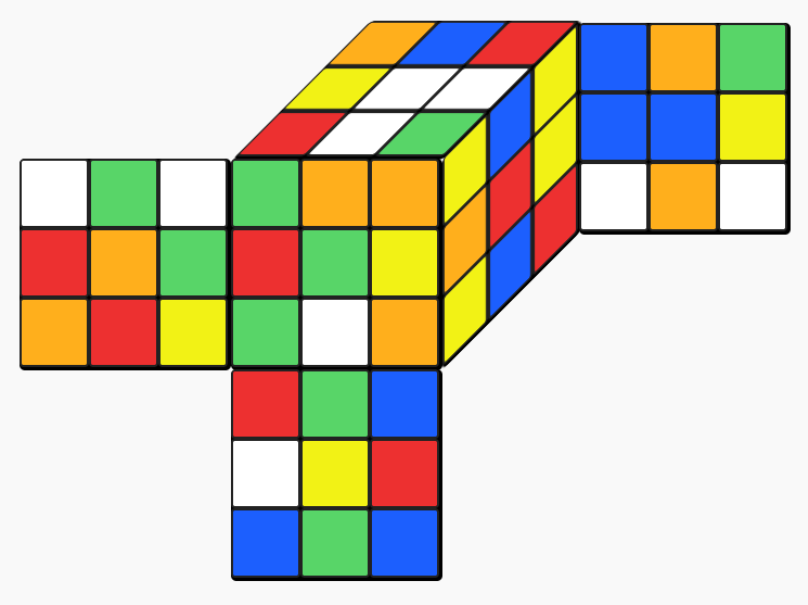
6. You must first test the movement without any cube and at very low speed to verify the placement of the clamp before manipulating a cube with the robot.

Now that everything is clear, you can fill all the missing parts in the dictionary `details_movement`.

In the following steps we are going to create a function that will be called to search the right movement in this dictionary. To be able to do that more easily we are going to need a second dictionary called `possible_movement`, the base is already in your code, you can complete it by following the same logic.

3. Step 3 : Cube resolution

For this example, we are going to use a scrambled cube that looks like this :



In python it is written like this :

```
scrambled_cube = {
                 'U': ['O', 'B', 'R', 'Y', 'W', 'W', 'R', 'W', 'G'],
                 'D': ['R', 'G', 'B', 'W', 'Y', 'R', 'B', 'G', 'B'],
                 'F': ['G', 'O', 'O', 'R', 'G', 'Y', 'G', 'W', 'O'],
                 'L': ['W', 'G', 'W', 'R', 'O', 'G', 'O', 'R', 'Y'],
                 'B': ['B', 'O', 'G', 'B', 'B', 'Y', 'W', 'O', 'W'],
                 'R': ['Y', 'B', 'Y', 'O', 'R', 'Y', 'Y', 'B', 'R']
                 }
```

To write how the cube looks you must follow some rules :

- At the beginning, the green face is always in front, and the white face is always on top
- When you write the face orange, green, red or blue, the cube must be oriented in the way so that the white is on top and the yellow is on the bottom and the face you want to write should face you
- When you write the white face, it should be facing you with the blue face on top
- When you write the yellow face, it should be facing you with the green face on top
- You write the color from the top left corner to the bottom right corner of the face, line by line

The function we will use is already implemented in python, so we will not have to create it. You can try to write one by yourself, but it will need something like AI and deep learning which is much harder to code.

The library where our function is, is called `kociemba` and we are going to use the function `kociemba.solve()`.

The first thing to do is to install it, type `pip install kociemba` in the anaconda prompt window and let it run. When it's done, you can return to your spyder window.

This function takes a scrambled cube and calculates a solution but our way of displaying the way our cube looks is understandable by the `kociemba.solve()` function. So, we need to translate it.

The `kociemba.solve()` function take a 54 characters long string representing the entire cube. Each character represents a color on each face. The faces are in a very particular order Up, Right, Face, Down, Left and then Back. Furthermore, this function does not take color as arguments but placement, so you must translate it so that the white face is represented as the upper face, the yellow face should be the down face, the green face should be the front face, ...

You must now create a function capable of translating our `scrambled_cube` in a way so that the `kociemba.solve()` function can understand it.

In our example, we start with the previous display and then our function will translate it to look like this :

```
In [15]: conversion(scrambled_cube)
Out[15]: 'LBRDUURUFDBDLRDDBRFLLRFDFULRFBUDRBFBUFURLFLRDBLFBBDULU'
```

Afterwards, after using `kociemba.solve()`, we will be left with a string that will be our final solution before using the memory function, which looks like this for the previous example :

```
In [19]: kociemba.solve(my_cube)
Out[19]: "R L2 F U' R2 U' D B' U R L' D2 R2 U' D2 L2 D L2 B2 D' B2"
```

The last function we will create is once again to convert what we have, to something our code can understand. The goal here is to obtain a list where each slot represents a movement, our example will look like this :

```
In [27]: corrected_solution = correction(kociemba.solve(my_cube))
    ...: print(corrected_solution)
['R', 'L2', 'F', "U'", 'R2', "U'", 'D', "B'", 'U', 'R', "L'", 'D2', 'R2', "U'", 'D2', 'L2', 'D', 'L2', 'B2', "D'", 'B2']
```

The function must take a string and return a list.

Lastly, we need to modify the solution to adapt it to the movement and orientation the robot will apply to the cube. This is where our previous function called `memorization()` will be used. You can simply call it after the previous function.

For our example the solution after memorization looks like this :

```
['R', 'U2', 'F', "F'", 'F2', "B'", 'U', "R'", 'L', 'F', "U'", 'B2', 'F2', "F'", 'U2', 'R2', 'B', 'L2', 'L2', "L'", 'R2']
```

4. Step 4 : Script Generation and File Export

The last step is to write the instructions into a USB stick that we can plug into the teach pendant so that the robot can read our program. We are going to use the section called "*Writing to the file*" in our python code.

For each movement in our solution, the python code must :

- Understand what type of movement it is (which face ? single or double rotation ? clockwise or counterclockwise ?)
- Call the right movement using the dictionaries `possible_movement` and `details_movement`
- Write each line of the instructions in the file we have chosen

You can complete the section so that the python code writes our UR-Script for the robot.

Testing Session

1. Code

To test our code and especially our UR-Script, we are going to bypass the resolution from before. For that, you are going to comment the line where you implemented the memorization to your solution and add another line where you say that the solution is what movement you want to test. It can for example be something like this if you want to test an **R movement** :

```
#robot_solution = memorisation(calculated_solution)
robot_solution = ["R"] # Test line
```

By doing so, the code will think that the solution is only an **R movement** ; to test other movement you can simply replace "R" by something else.

We know that the code works if it writes the instructions we want in the correct file and order.

After testing it with one movement, you can try to run it with three movements. Let's first try with this sequence of movements : `["L", "B", "U'"]`. It should normally be this sequence written in the file.

To test the memory, we can move our test line before applying the memory function, like this :

```
#calculated_solution = correction(kociemba.solve(my_cube))
calculated_solution = ["L", "B", "U'"] # Test line

# We modify the solution to integrate the memory of the robot
robot_solution = memorisation(calculated_solution)
```

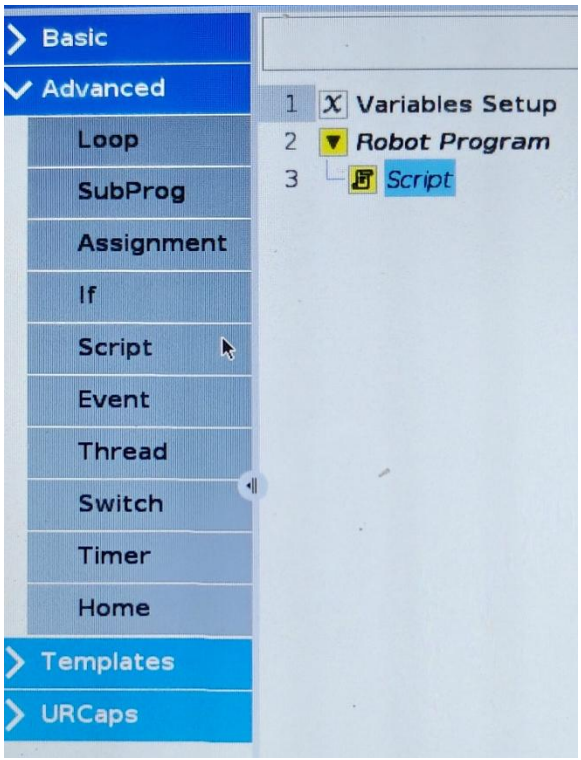
`robot_solution` should be equal to this `['L', 'F', "B'"]`.

2. Robot

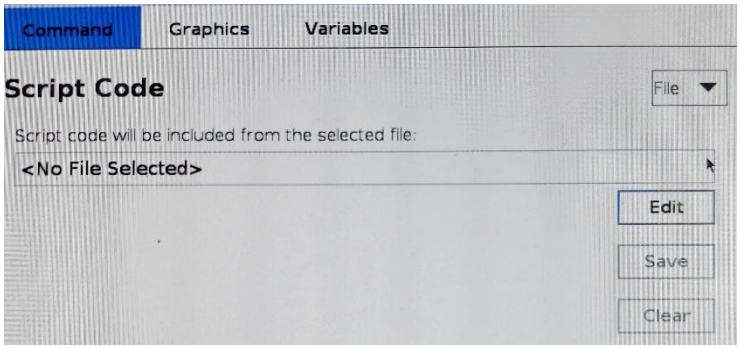
To make the robot follow the program we just wrote we will use the USB stick where the program is located. To plug a USB stick into the robot we will use the teach pendant, it can be plugged here (step 1) :



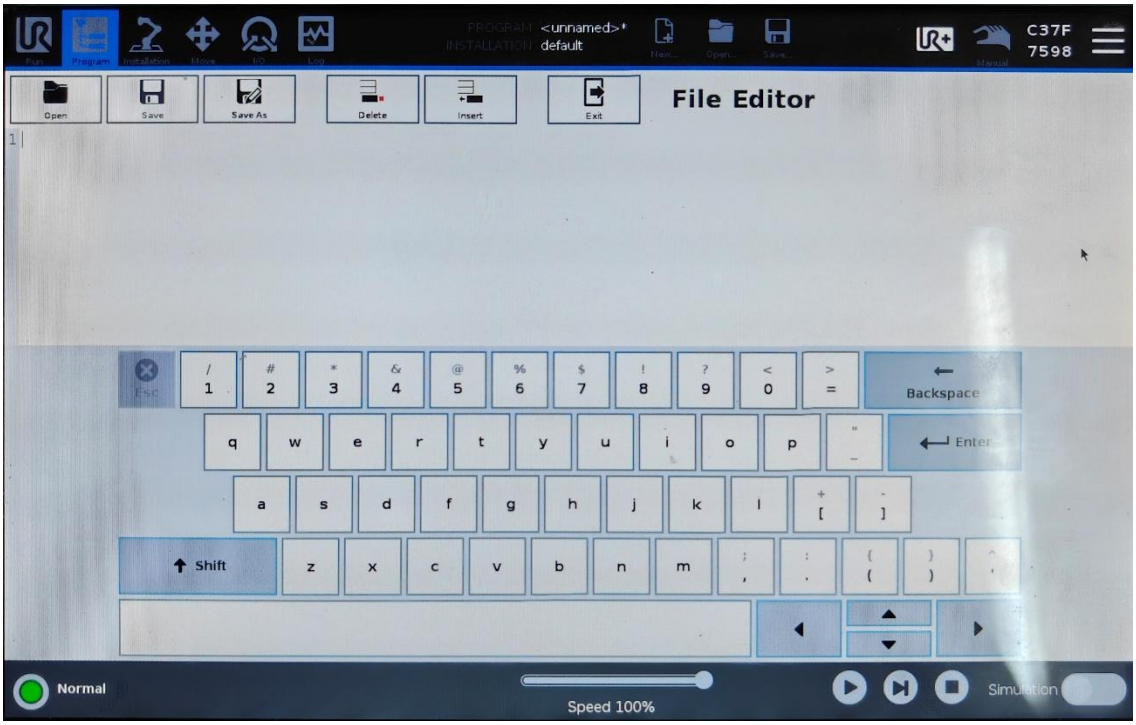
When it's done, you can go to the "*Program*" window, choose the "*Advanced*" menu and click on "*Script*" in an empty robot program, be sure to disable the option to make the program loop indefinitely (step 2) :



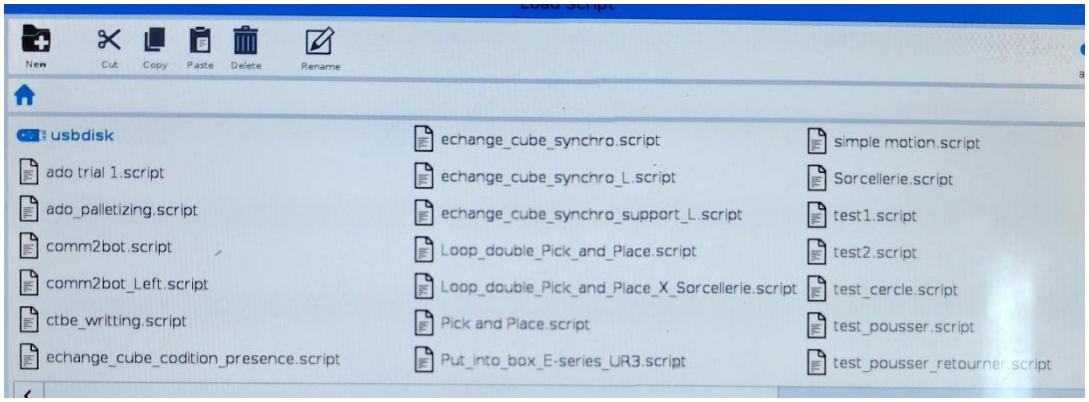
After clicking on "*Script*" in the tree structure of the program you will face a screen like this one, choose "*Line*" on the right then "*File*" to be on this screen (step 3) :



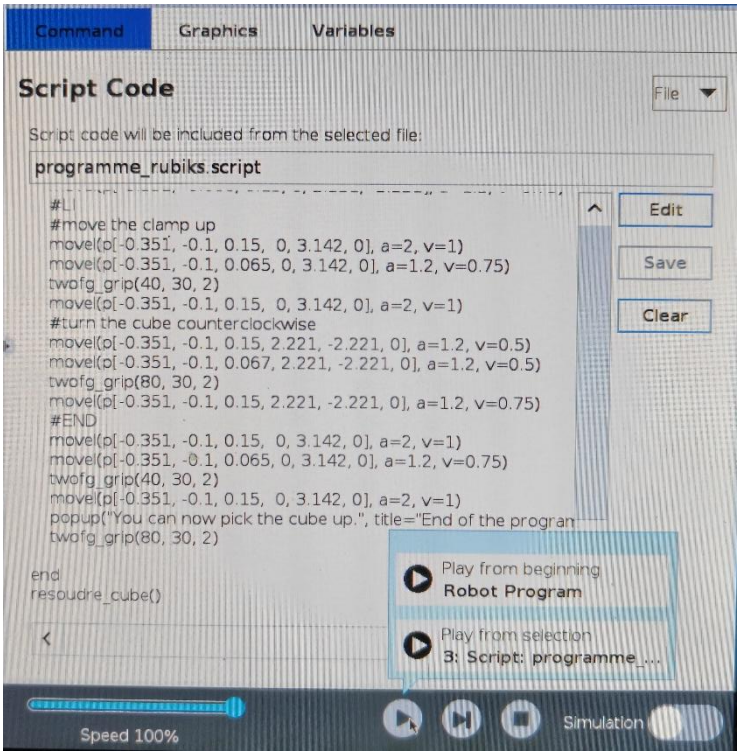
After choosing to edit the script you will enter the "File Editor" mode on the following screen (step 4):



To open the UR-Script you want to test, you must choose the "*Open*" option above, then choose the file called "*usbdisk*" and search your program in it and click "*Open*" on the bottom right :



You can then exit the “File Editor” mode and run your program :



You can now try every movement at very low speed and without the cube, to be sure that the position of the clamps is good around it. You can check the position at several moments by stopping the program and placing the cube inside the base but don’t use the cube when the robot is moving.

When everything looks good, you can try some basic movements like “D”, “D’” or “D2” with the cube to verify the rotation. Then finally all the other movement with the cube but always at low speed.

To go further

À compléter